

Name: Demarce Williams

Class: Signals and Systems

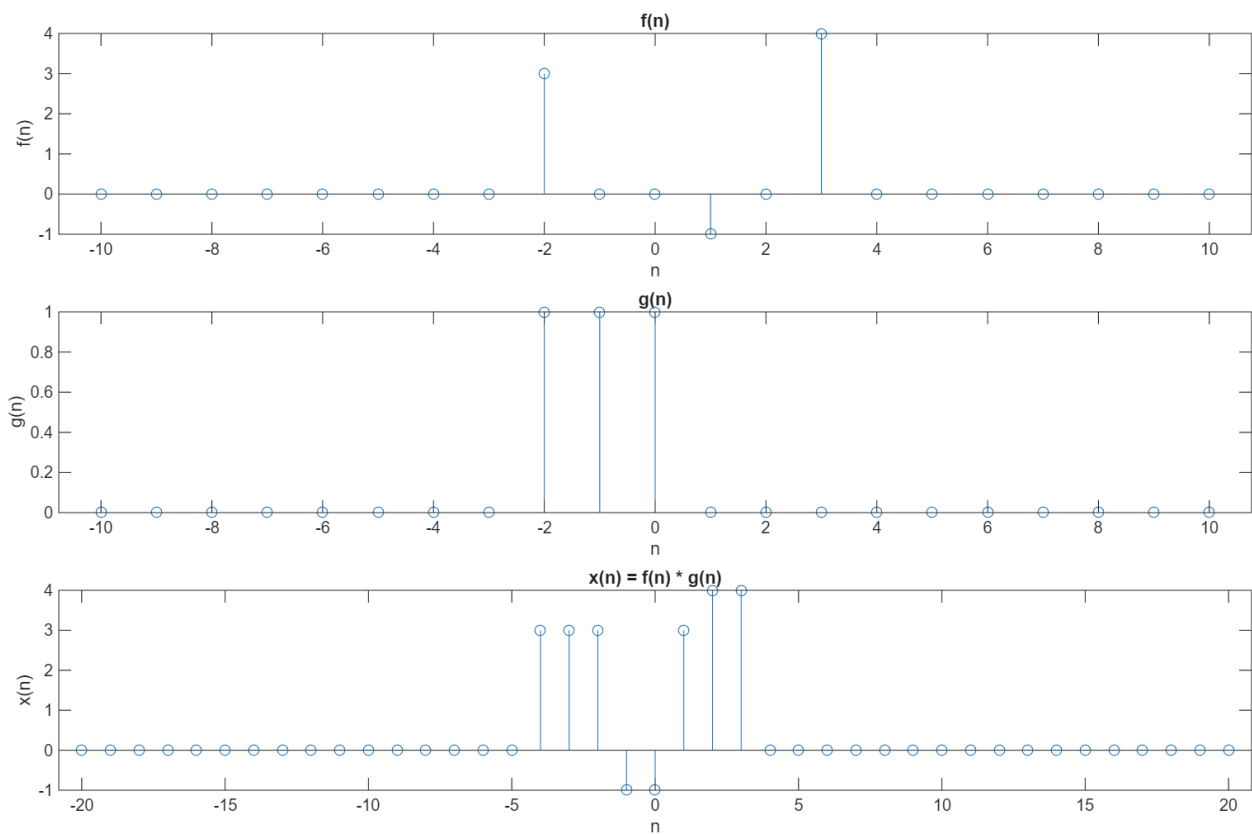
Lab #: 03

Date: September 18, 2025

Lab 2: Smoothing Data and Difference Equations

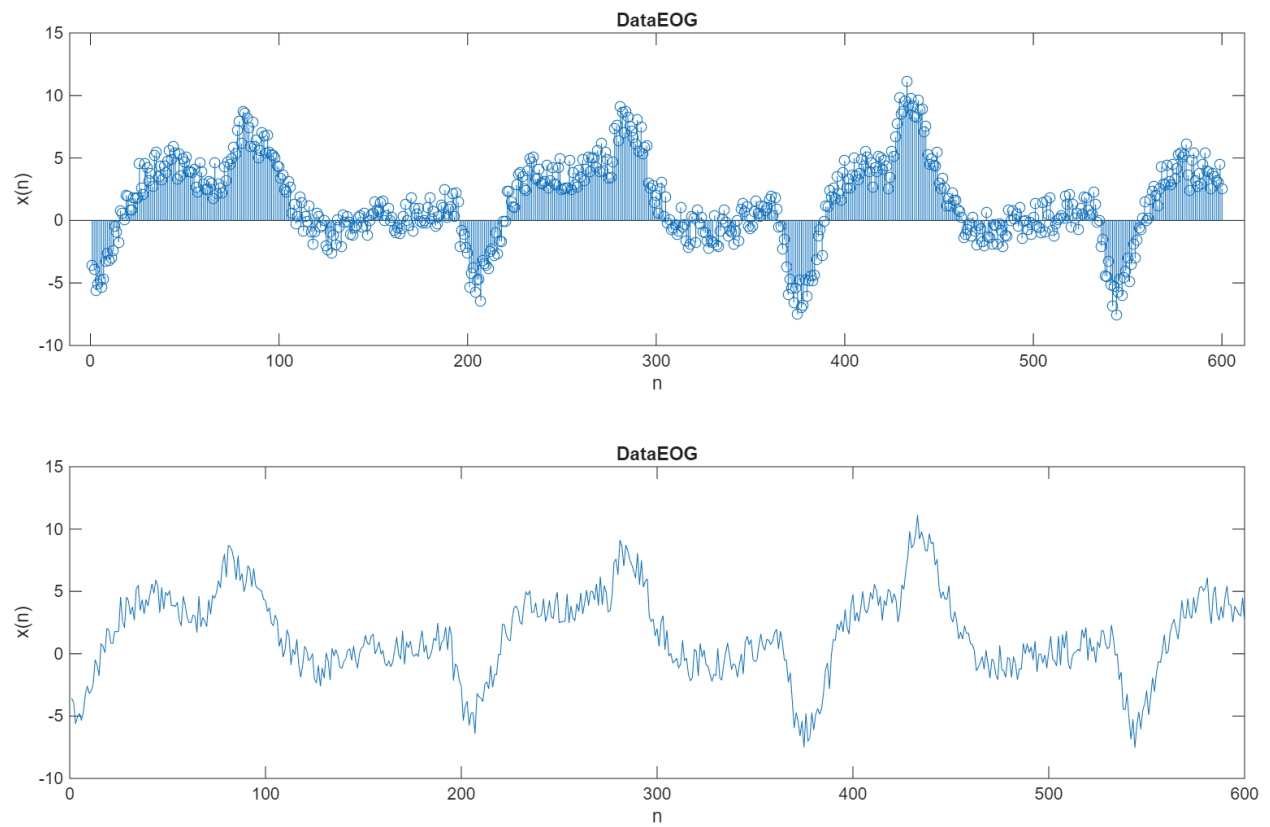
Solutions for Tasks 1–3

Task 1: Convolution of Non-Causal Signals



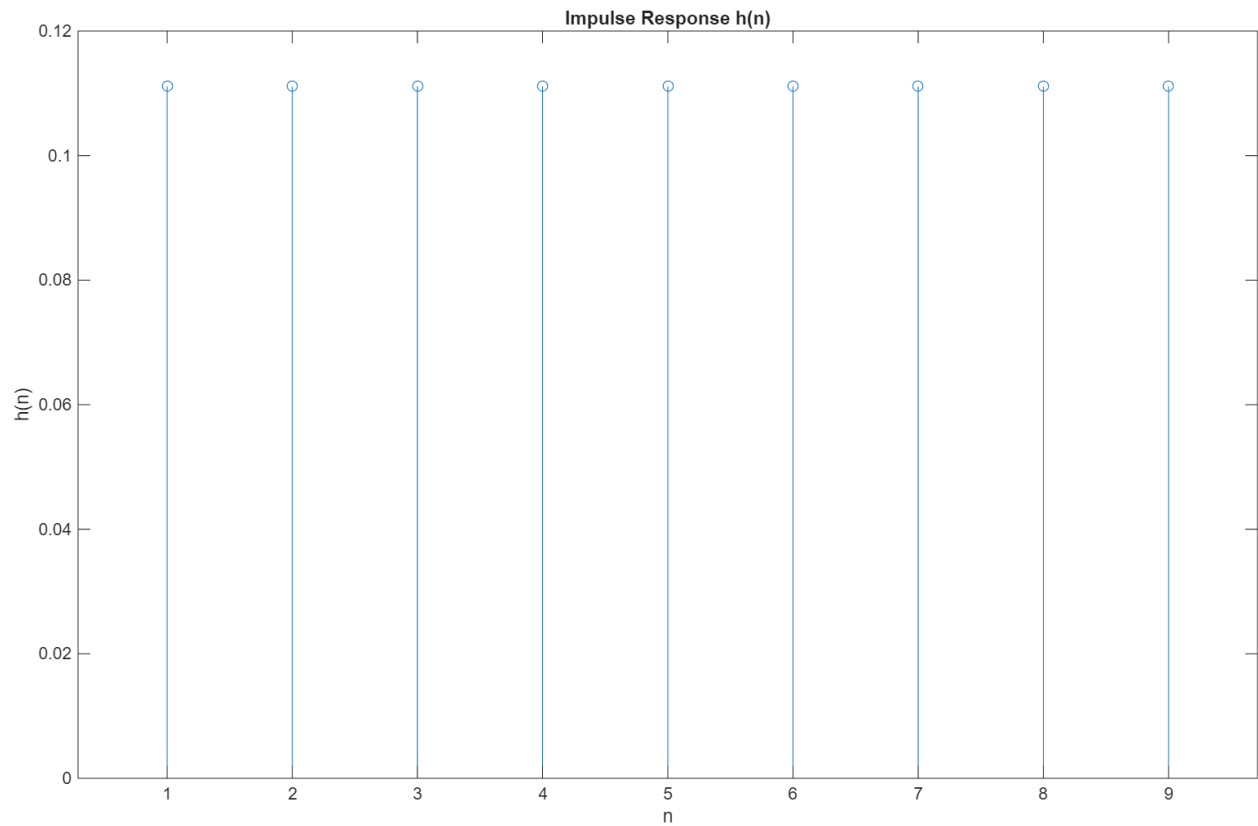
The convolution "spreads out" each delta in $f(n)$ into shifted and scaled copies of $g(n)$.

Task 2: Smoothing Data

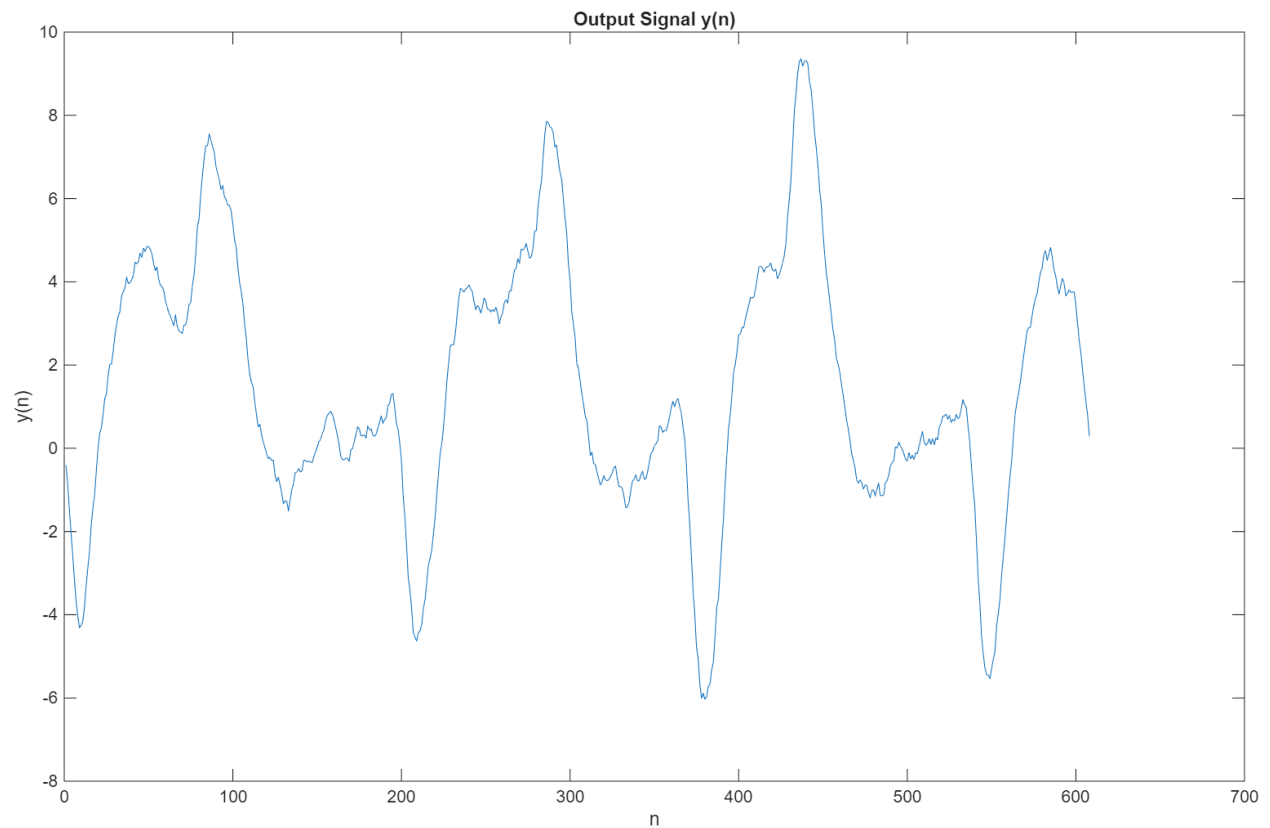


The EOG data $x(n)$ is noisy and oscillatory.

Stem plot looks cluttered due to many points while the line plot provides better visualization.



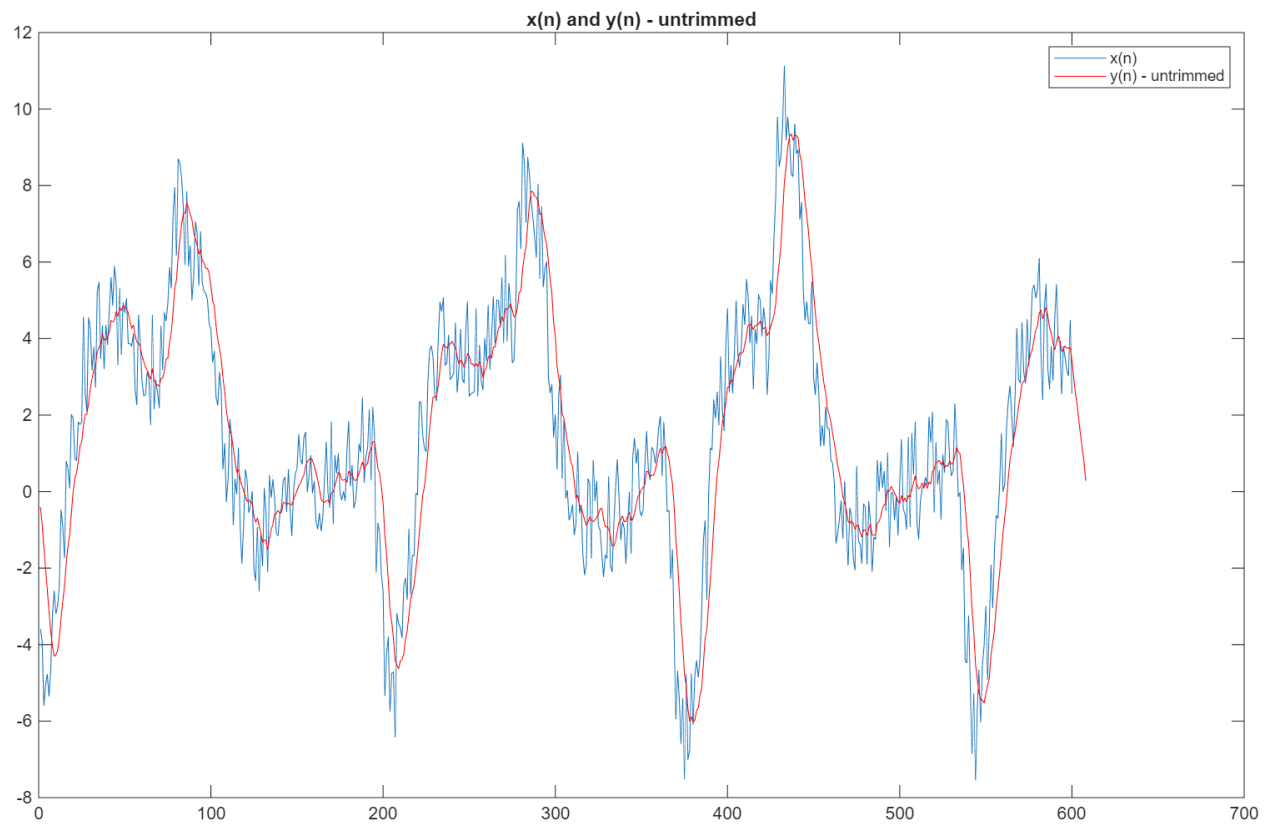
$h(n)=1/9[1,1,...,1]$ (length 9 moving average). This is a simple low-pass filter that averages neighboring samples.



Output $y(n)=x(n)*h(n)$ is a smoothed version of the input.

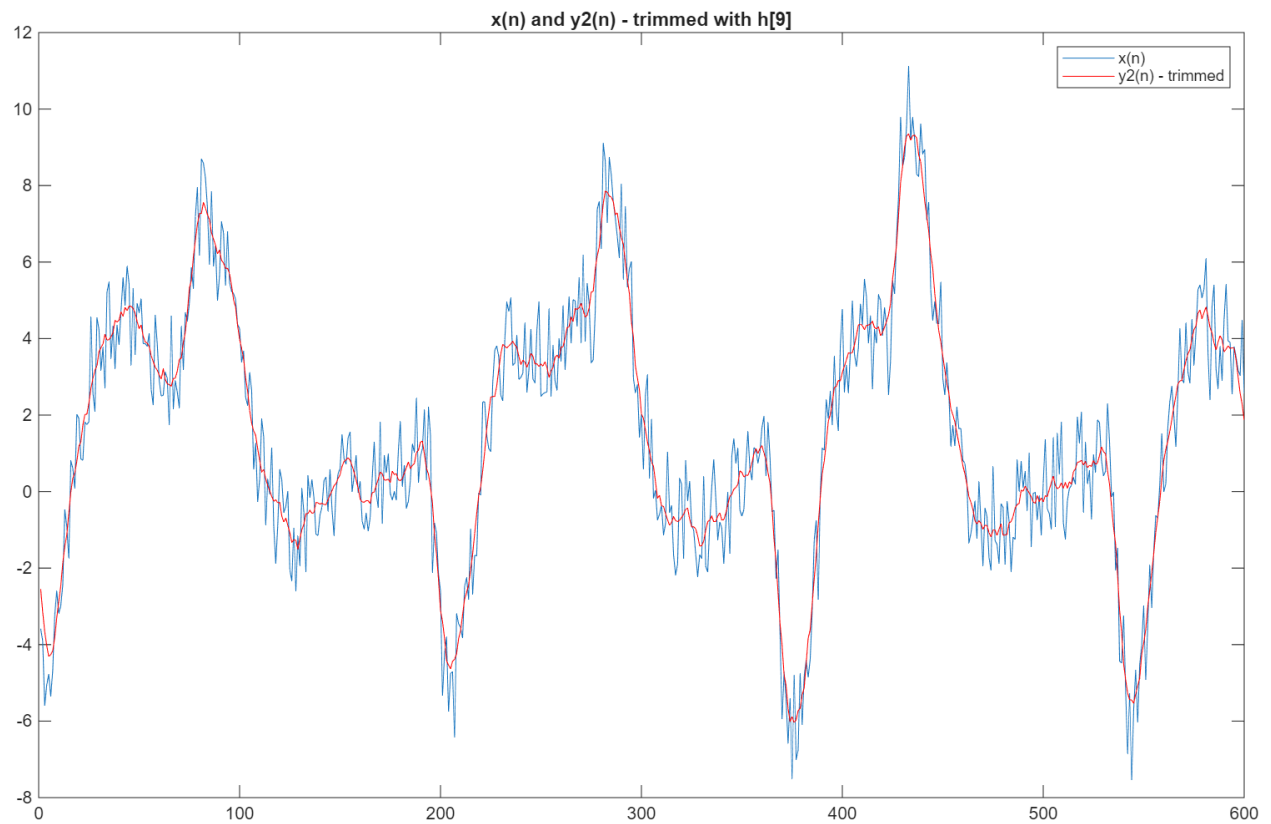
Rapid fluctuations (noise) are suppressed, while overall trends remain.

Length of Output: $\text{length}(y)=\text{length}(x)+\text{length}(h)-1$.



$y(n)$ is longer than $x(n)$, and initially misaligned.

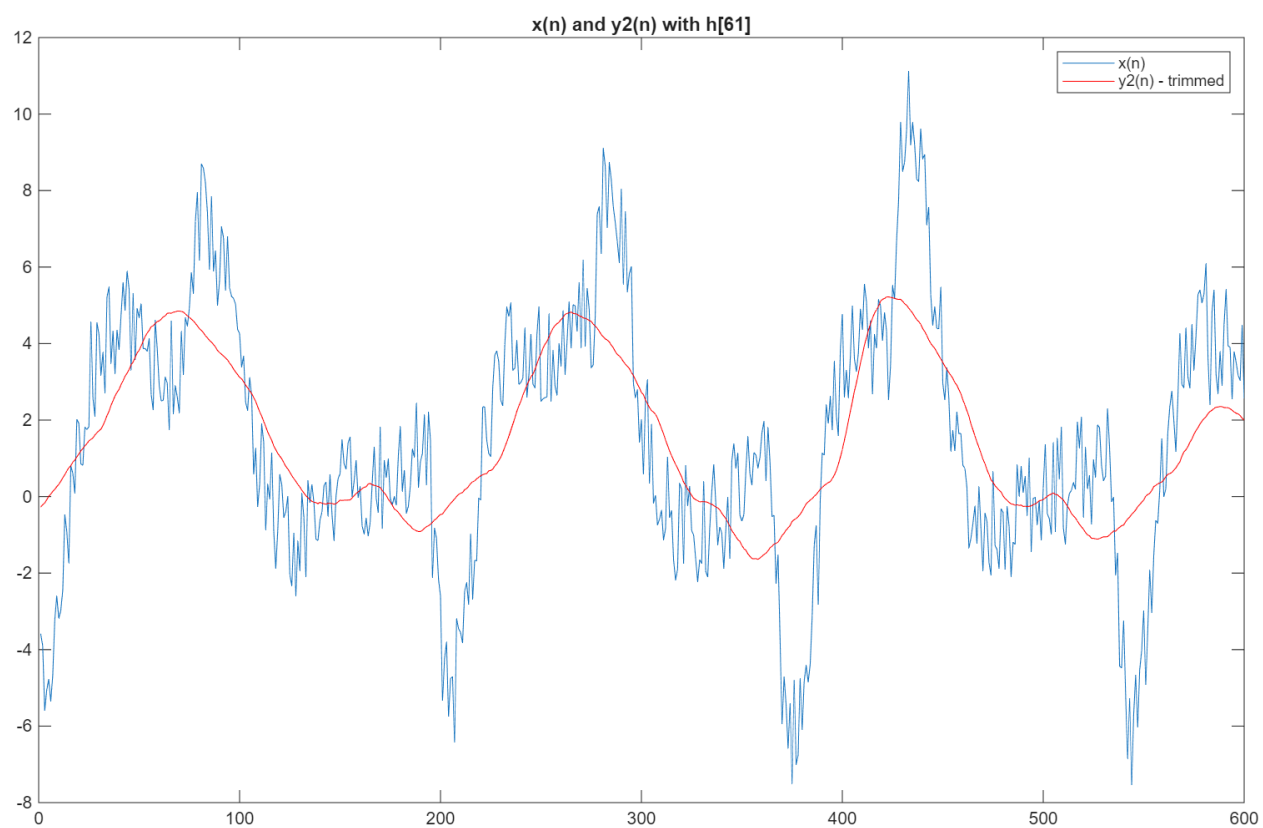
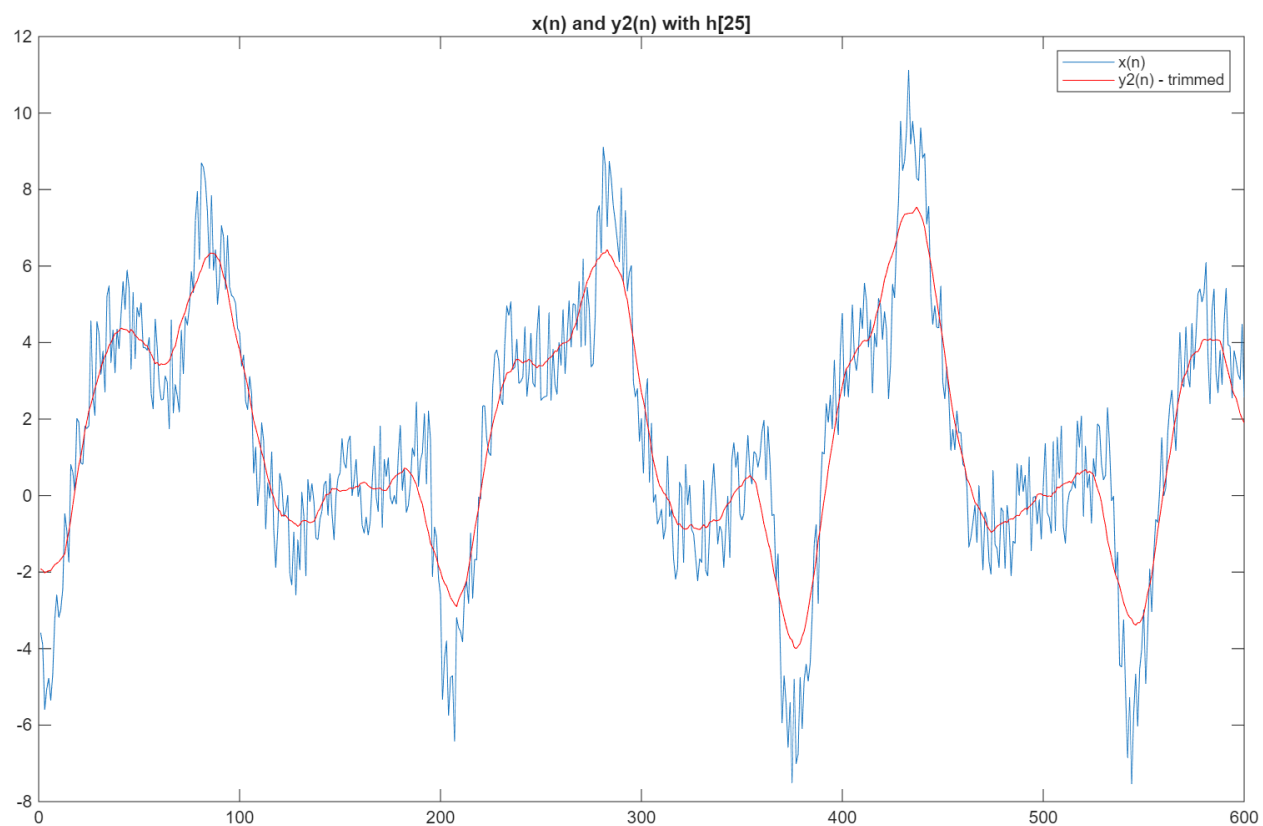
This happens because convolution includes edge effects.



Removing 4 samples at the beginning and end aligns $y2(n)$ with $x(n)$.

Length of $y2$ now equals length of $x(n)$.

Plot shows that $y2(n)$ is a smoothed version aligned with the original signal.



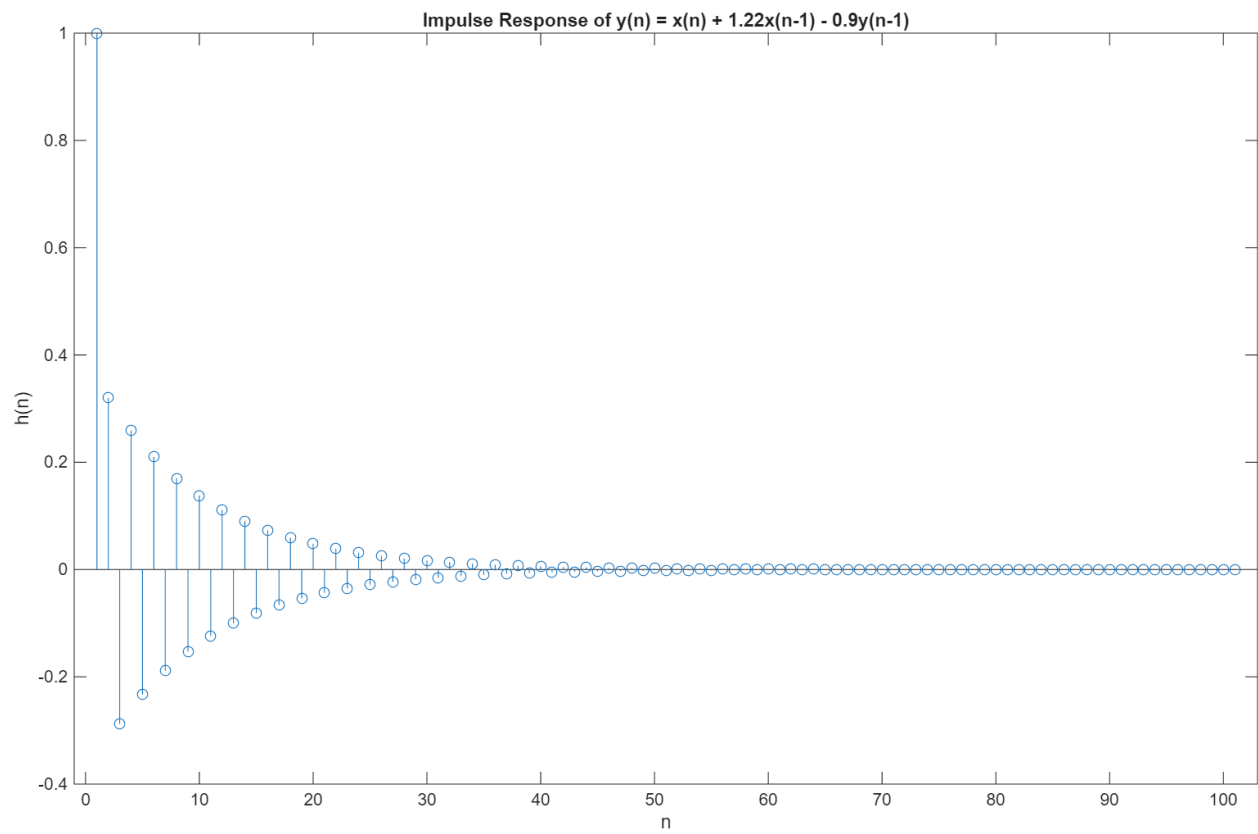
With $h=1/25[1, \dots, 1]$, trimming requires 12 samples on each side.

With $h=1/61[1, \dots, 1]$, trimming requires 30 samples on each side.

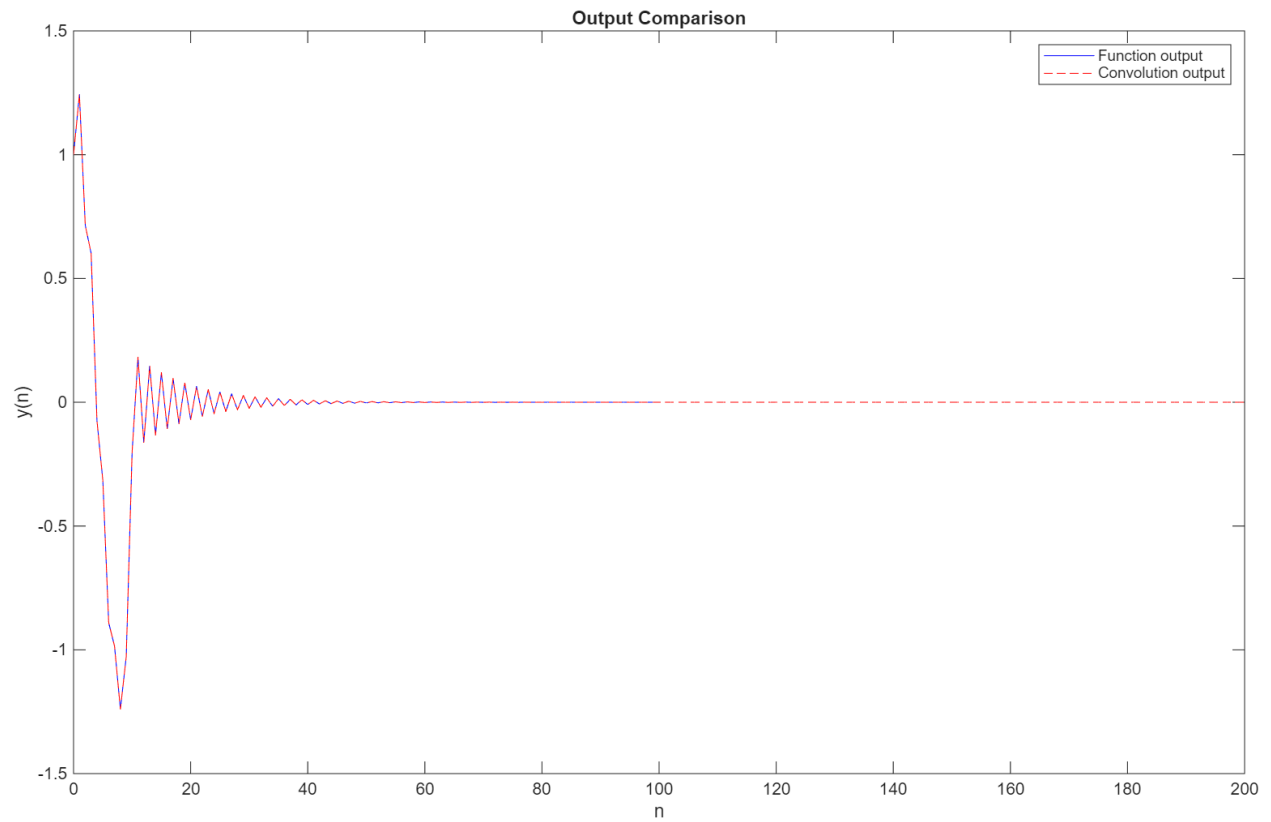
Observations:

1. Increasing filter length results in smoother output (noise further attenuated).
2. Peaks are more flattened, and details are lost.
3. Edge effects become stronger (larger delays, more trimming needed).

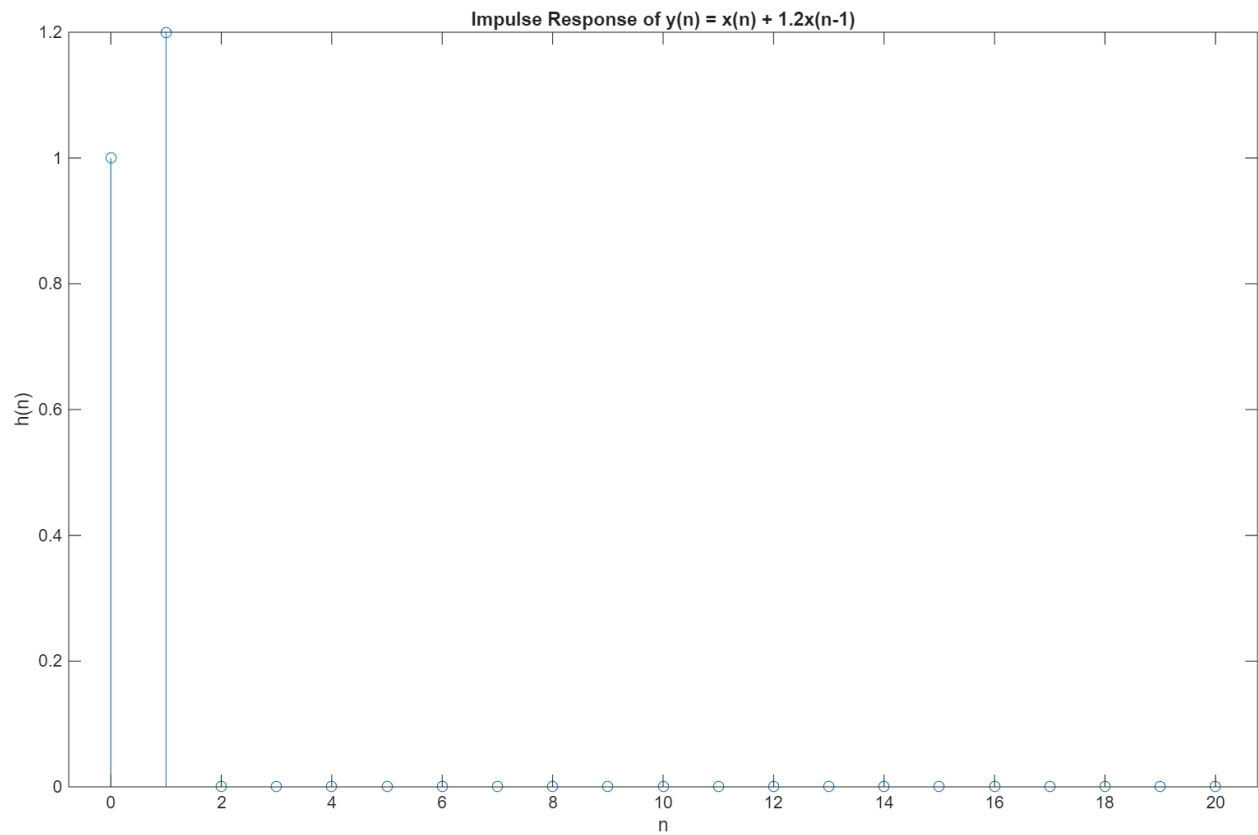
Task 3: Difference Equations



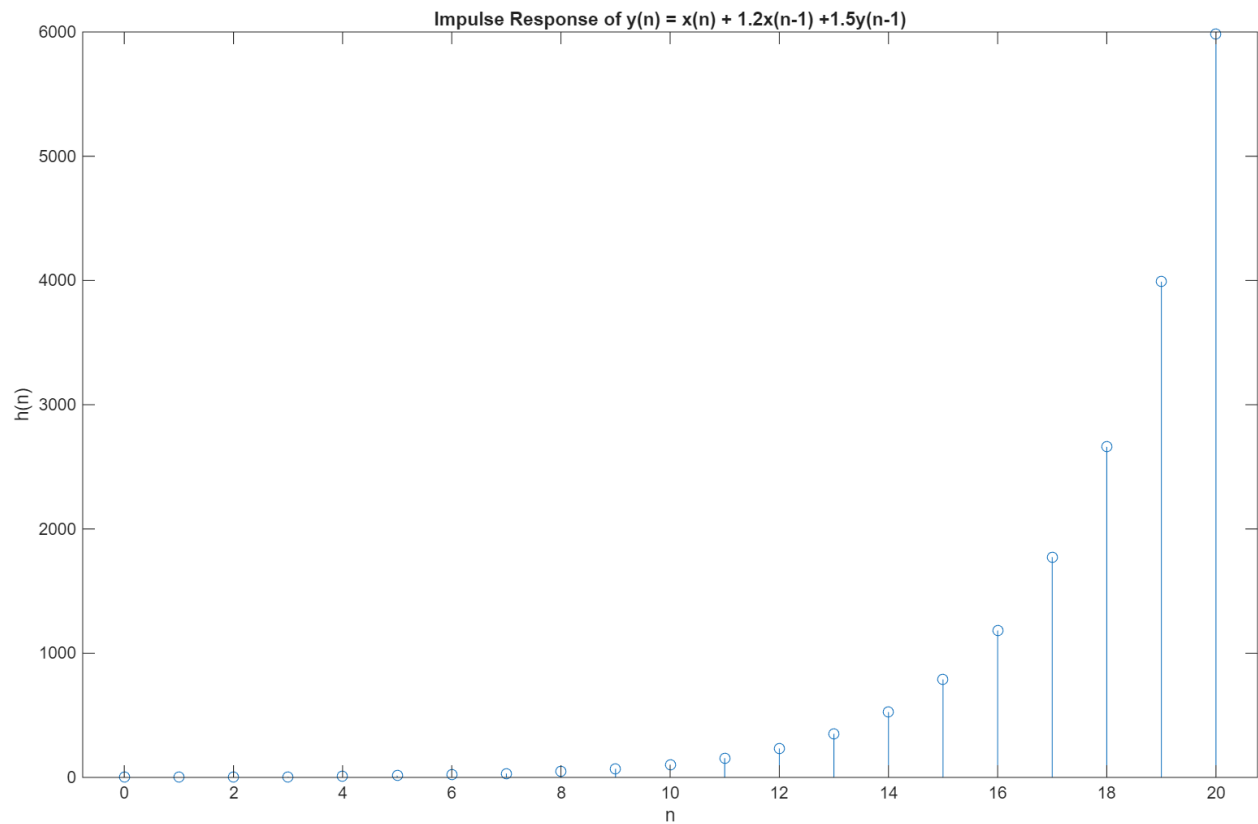
Impulse response decays over time, oscillatory because of the negative feedback term $-0.9y(n-1]$.



Using convolution $y(n)=h(n)*x(n)$ and direct evaluation with *mydiffEq* gives the **same output**, confirming the correctness of both approaches.



Impulse response is simply two nonzero values: $[1, 1.2]$. Since there is no feedback, response ends after 2 samples.



Impulse response grows in magnitude because of positive feedback term $+1.5y(n-1)$.

This system is **unstable** compared to the others because impulse response does not decay, instead it diverges. Compared to the first two: instead of settling, it blows up.

Matlab Code

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% Name: Demarce Willaims
```

```
% Class: Signals and Systems
```

```
% Lab #: 03
```

```
% Date: September 18, 2025
```

```
% Lab 2: Smoothing Data and Difference Equations
```

```
% Solutions for Tasks 1–3
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
%  
%
```

```
close all; clear; clc;
```

```
%% Task 1: Convolution of Non-Causal Signals
```

```
figure('Name','Task 1: Convolution of Non-Causal Signals');
```

```
n = -10:10; % Time vector for f and g
```

```
%  $f(n) = 3\delta(n+2) - \delta(n-1) + 4\delta(n-3)$ 
```

```
subplot(3,1,1);
```

```
f = 3*(n==-2) - (n==1) + 4*(n==3);
```

```
stem(n, f);
```

```
title('f(n)');
```

```
xlabel('n'); ylabel('f(n)');
```

```
%  $g(n) = u(n+2) - u(n-1)$ 
```

```
subplot(3,1,2);  
g = (n >= -2) - (n >= 1);  
stem(n, g);  
title('g(n)');  
xlabel('n'); ylabel('g(n)');
```

```
%  $x(n) = f(n) * g(n)$   
x_fg = conv(f, g); % Linear convolution  
% Convolution range: from  $n_{\text{start}} = n(1) + n(1)$  to  $n_{\text{end}} = n(\text{end}) + n(\text{end})$   
nx = (2*n(1)) : (2*n(end)); % -40 to 40 in this case
```

```
subplot(3,1,3);  
stem(nx, x_fg);  
title('x(n) = f(n) * g(n)');  
xlabel('n'); ylabel('x(n)');
```

```
%% Task 2: Smoothing Data
```

```
figure('Name','Task 2: Smoothing Data');
```

```
x = load("DataEOG.txt") ;
```

```
n = 1:length(x);
```

```
subplot(2,1,1);  
stem(n, x);  
title('DataEOG');  
xlabel('n'); ylabel('x(n)');
```

```
subplot(2,1,2);  
plot(n, x);  
title('DataEOG');  
xlabel('n'); ylabel('x(n)');
```

```
%Impulse Response for an LTI system
```

```
h = ones(1,9)/9; % moving average filter (length 9)
```

```
% Stem plot of impulse response  
n = 1:length(h);  
figure('Name','Impulse Response');  
stem(n, h);  
title('Impulse Response h(n)');  
xlabel('n'); ylabel('h(n)');
```

```
% Convolution of input with impulse response  
y = conv(x, h);
```

```
% Plot output signal y(n)  
figure;  
plot(y);  
title('Output Signal y(n)');  
xlabel('n'); ylabel('y(n)');
```

```
% (a) Effect of convolution:
```

```
% The output y(n) is a smoothed version of x(n),  
% where each value is the average of the previous 9 samples.
```

```
% (b) Length of y(n):
```

```
% length(y) = length(x) + length(h) - 1
```

```
% (c) Plot x(n) and y(n) together
```

```
figure;
```

```
plot(x); hold on;
```

```
plot(y,'r');
```

```
legend('x(n)', 'y(n) - untrimmed');
```

```
title('x(n) and y(n) - untrimmed');
```

```
% y(n) is longer than x(n) and misaligned due to convolution edge effects.
```

```
% (d) Trim y(n) to align with x(n)
```

```
y2 = y;
```

```
y2(1:4) = []; % remove first 4 samples
```

```
y2(end-3:end) = []; % remove last 4 samples
```

```
% Plot x(n) and y2(n) together
```

```
figure;
```

```
plot(x); hold on;
```

```
plot(y2,'r');
```

```
legend('x(n)', 'y2(n) - trimmed');
```



```
title('x(n) and y2(n) - trimmed with h[9]');
```

```
% y2 is now aligned with x(n), smoothing effect is visible.
```

```
% Part (e) Use h = ones(1,25)/25
```

```
h = ones(1,25)/25;
```

```
y = conv(x,h);
```

```
M = length(h);
```

```
d = (M-1)/2;           % samples to trim for alignment
```

```
y2 = y;
```

```
y2(1:d) = [];
```

```
y2(end-d+1:end) = [];
```

```
% Plot
```

```
figure;
```

```
plot(x); hold on;
```

```
plot(y2,'r');
```

```
legend('x(n)','y2(n) - trimmed');
```

```
title('x(n) and y2(n) with h[25]');
```

```
% Part (f) Use h = ones(1,61)/61
```

```
h = ones(1,61)/61;
```

```
y = conv(x,h);
```

```
M = length(h);
```

```
d = (M-1)/2;           % samples to trim for alignment
```

```

y2 = y;
y2(1:d) = [];
y2(end-d+1:end) = [];

% Plot
figure;
plot(x); hold on;
plot(y2,'r');
legend('x(n)', 'y2(n) - trimmed');
title('x(n) and y2(n) with h[61]');

```

```

% Observations:

```

- % 1. Increasing h length smooths x(n) more (less noise, peaks flattened).
- % 2. Longer filters introduce more edge effects and larger delays (trim more samples).
- % 3. Alignment requires trimming to match x(n) length.

```

%% Task 3: Difference Equations

```

```

% Difference Equation 1

```

```

% Impulse input

```

```

x = [1, zeros(1,100)];

```

```

y = mydiffeq(x);

```

```

% Plot impulse response

```

```

figure;

```

```

stem(y);

```

```
title('Impulse Response of  $y(n) = x(n) + 1.22x(n-1) - 0.9y(n-1)$ ');  
xlabel('n');  
ylabel('h(n)');
```

```
% (1) Convolution with impulse response
```

```
n = 0:100;  
x = cos(pi*n/8).*((n>=0) -(n>=10));  
y_conv = conv(y, x); % y from part (a) is h(n)
```

```
% (2) Using the function directly
```

```
y_func = mydiffeq(x);
```

```
% Plot comparison
```

```
figure;  
plot(0:length(y_func)-1, y_func, 'b'); hold on;  
plot(0:length(y_conv)-1, y_conv, 'r--');  
legend('Function output','Convolution output');  
title('Output Comparison');  
xlabel('n'); ylabel('y(n)');
```

```
% Difference Equation 2
```

```
% Impulse input
```

```
x = [1, zeros(1,20)];  
y = mydiffeq2(x);
```

```
% Plot impulse response
```

```
figure;  
stem(0:20, y);  
title('Impulse Response of  $y(n) = x(n) + 1.2x(n-1)$ ');  
xlabel('n');  
ylabel('h(n)');
```

% Difference Equation 3

```
% Impulse input  
x = [1, zeros(1,20)];  
y = mydiffeq3(x);
```

% Plot impulse response

```
figure;  
stem(0:20, y);  
title('Impulse Response of  $y(n) = x(n) + 1.2x(n-1) + 1.5y(n-1)$ ');  
xlabel('n');  
ylabel('h(n)');
```

```
function y = mydiffeq(x)

N = length(x);
y = zeros(1,N); % initialize output

for n = 1:N
    if n==1
        y(n) = x(n); % x(n-1) does not exist for the first entry n = 1
    else
        y(n) = x(n) + 1.22*x(n-1) - 0.9*y(n-1);
    end
end
end
```

```
function y = mydiffeq2(x)

N = length(x);
y = zeros(1,N); % initialize output

for n = 1:N
    if n==1
        y(n) = x(n); % x(n-1) does not exist for the first entry n = 1
    else
        y(n) = x(n) + 1.2*x(n-1);
    end
end
end
```

```
function y = mydiffeq3(x)

N = length(x);
y = zeros(1,N); % initialize output

for n = 1:N
    if n==1
        y(n) = x(n); % x(n-1) does not exist for the first entry n = 1
    else
        y(n) = x(n) + 1.2*x(n-1) + 1.5*y(n-1);
    end
end
end
```